Final Project

Svelte Seminar 2026

Estimated effort: ~14 hours | **Deadline: April 16, 2026**

Requirements

Every project must include:

- **Landing page** — a clear entry point that explains what the app does
- **At least one additional page** — meaningful navigation, not empty routes
- **Custom styling** — should look intentionally designed (Tailwind recommended but not required)
- **Authentication** — login flow that gates at least one feature
- **Supabase** — for authentication (Lane B) or authentication + data storage (Lane A)
- **Hosted on Vercel** — a live URL that works

Choose Your Lane

Pick **one** lane. Both are equally valid — what matters is that the chosen approach is executed well.

	Lane A: Server-Side MPA	Lane B: Interactive Client
Focus	Backend data management	Frontend state & reactivity
Core pattern	Load functions + form actions	`$state` + `$derived`
Database	Supabase for auth + data	Supabase for auth only
Complexity lives in	Data flow & CRUD operations	Interconnected reactive state

Lane A: Server-Side MPA

Traditional multi-page application with proper backend data management.

- **Load functions** (`+page.server.ts`) fetch data from Supabase on the server
- **Form actions** handle create, update, and delete operations
- **Full CRUD** — users can create, read, update, and delete their data
- **Server-rendered pages** — content is in the HTML before JavaScript loads
- Database design matters: think about your tables, relationships, and row-level security

What we're looking for: clean data flow from database → load function → page → form action → database. Multiple related operations (not just "add to list").

Lane B: Interactive Client

A reactive, stateful client-side application. Does not need a database beyond auth — the complexity lives in the UI logic.

- **Multiple** `$state` **values** that represent different aspects of the app's state
- `$derived` **values** that compute things from state (totals, filters, streaks, progress, validation)
- **Rich interactivity** — the UI responds immediately to user actions without round-trips to the server
- State relationships matter: changes to one piece of state should cascade through derived values

What we're looking for: meaningful use of reactive state — not just one counter, but interconnected state where `$derived` does real work.

Lane A Ideas

Project	What it does	CRUD operations
Recipe book	Browse, submit, and search recipes	Create/edit/delete recipes, tag by category
Movie/book reviews	Browse titles, write and read reviews	Add titles, write/edit/delete reviews
Bookmark manager	Save, tag, and organize links	Add/edit/delete bookmarks, manage tags
Class notes	Create and browse markdown notes by subject	Create/edit/delete notes, organize by subject
Event board	Post and browse upcoming events	Create/edit/delete events, RSVP

Lane B Ideas

Project	What it does	State & `$derived` usage
Pomodoro timer	Work/break cycles with session tracking	Time remaining, session count, break logic, daily stats
Quiz app	Answer questions, track progress	Score, progress percentage, streaks, difficulty adjustment
Workout planner	Build routines from an exercise library	Total volume, duration, set counts, rest timers
Flashcard app	Study mode with spaced repetition	Mastery score, cards due, deck progress, review intervals
Budget calculator	Plan income vs expenses	Category totals, balance, savings rate, charts

What Makes a Project the Right Size?

- **Too small:** a single page with a form and a list (that's an exercise, not a project)
- **Right size:** 3–4 pages, one clear data model, auth that makes sense for the use case
- **Too big:** multiple user roles, real-time features, file uploads, payment integration

A simple project done well will score higher than an ambitious project that's half-broken.

Grading

Score	Grade
100–90	1
89–75	2
74–60	3
59–50	4
< 50	5

Grading is holistic — **functionality, code quality, effort & ambition, creativity.**

Submission

- **Live URL** (Vercel)
- **Source code** (GitHub repository)
- **Deadline: April 16, 2026**